

Deep Reinforcement Learning

Module 4: Continuous Control and Modern Actor–Critic Methods

From the actor–critic bridge to DDPG and PPO

Mustafa Ozger

Aalborg University

24 March 2026

Why this module matters

- Module 3 ended with the **actor–critic bridge**: the actor chooses actions, and the critic evaluates them.
- That bridge becomes essential in **continuous control**, where the action is not chosen from a short discrete list.
- In many engineering systems, the action is a real-valued control signal: steering angle, torque, acceleration, throttle, transmit power, or bank angle.
- This module introduces two influential actor–critic families for that setting:
 - **DDPG**: deterministic, off-policy, replay-based continuous control.
 - **PPO**: stochastic, on-policy, clipped policy optimization.

Teaching goal

Students should leave this lecture understanding *why* these algorithms exist, *what each block in their architecture diagrams does*, and *how the actor and critic are updated*.

Learning outcomes

After this lecture, students should be able to:

- explain why continuous action spaces are hard for DQN-style maximization,
- explain the actor–critic idea in plain language,
- distinguish actor, critic, state value, action value, TD error, and advantage,
- explain deterministic versus stochastic policies,
- read the DDPG architecture diagram and explain every major block,
- derive the DDPG critic target and explain the actor update conceptually,
- explain why DDPG uses replay memory, target networks, and exploration noise,
- read the PPO architecture diagram and explain the role of old policy, ratio, advantage, value loss, and entropy,
- explain the clipped PPO objective at an intuitive level,
- compare DDPG and PPO as two different design choices within actor–critic RL.

- ① Why continuous control changes the problem
- ② Actor–critic from first principles
- ③ DDPG: deterministic, off-policy actor–critic
- ④ PPO: stochastic, clipped, on-policy actor–critic
- ⑤ Comparing DDPG and PPO

Notation used in this lecture

To keep notation consistent across actor–critic, DDPG, and PPO, we use:

- s_t : current state, a_t : action executed at time t
- r_{t+1} : reward observed after taking a_t , s_{t+1} : next state
- $\text{done}_t \in \{0, 1\}$: terminal indicator
- $\pi_\theta(a | s)$: stochastic actor / policy
- $\mu_\theta(s)$: deterministic actor
- $V_\psi(s)$: state-value critic
- $Q_\phi(s, a)$: action-value critic
- $\hat{A}_t$: advantage estimate used in policy updates

Important note for imported figures

The imported DDPG and PPO diagrams use a few different labels. Around those figures, we explicitly map their notation back to the lecture notation so the flow remains easy to follow.

Why continuous actions are different

DQN-style decision rule

$$a_t = \arg \max_a Q(s_t, a)$$

This is straightforward when the action set is small and discrete.

Discrete action example

Atari joystick:
left, right, up, down, fire.

Continuous action example

Steering angle $\in [-30^\circ, 30^\circ]$ or torque
 $\in [-2, 2]$.

For continuous actions, maximizing $Q(s, a)$ over all possible real-valued actions becomes a separate optimization problem at every decision step.

Why not just discretize the action space?

A natural first idea is to turn a continuous action into many discrete bins.

- Fine discretization creates many actions and increases computation.
- Multiple action dimensions cause a combinatorial explosion.
- The selected action may still be too coarse for smooth control.

Main consequence

In continuous control, it is often better to let a policy network **output the action directly** rather than search over many candidate actions.

From value-based control to actor-critic control

Value-based view

- Learn $Q(s, a)$.
- Choose actions by maximizing Q .
- Very natural for discrete action spaces.

Actor-critic view

- Learn a policy *and* a learned evaluator.
- The actor proposes actions directly.
- The critic tells the actor whether those actions were good.

Bridge from Module 3

Actor-critic combines direct policy learning with value-based feedback. That is why it is the key bridge from REINFORCE to practical continuous-control methods.

Actor-critic intuition

Actor

- represents the policy,
- chooses the next action,
- answers: *what should I do next?*

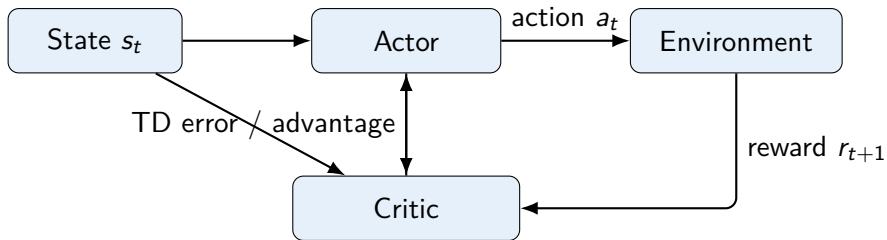
Critic

- estimates value or advantage,
- evaluates the result of the actor's decision,
- answers: *was that action better or worse than expected?*

Mental model

The actor is the decision-maker. The critic is the evaluator or teacher.

Generic actor–critic loop



- The actor proposes an action.
- The environment returns reward and next state.
- The critic turns that experience into a learning signal.
- The actor updates itself using that signal.

Two common critic choices

Value critic

$$V_{\psi}(s_t)$$

- estimates how good the current state is,
- common in PPO and many policy-gradient methods,
- often used to compute TD error or advantage.

Action-value critic

$$Q_{\phi}(s_t, a_t)$$

- estimates how good a specific action is in a state,
- common in DDPG,
- directly tells the actor whether its chosen action is promising.

Deterministic vs stochastic actors

Deterministic actor

$$a_t = \mu_{\theta}(s_t)$$

- outputs one action,
- natural for continuous control,
- exploration must be added externally,
- used by DDPG.

Stochastic actor

$$a_t \sim \pi_{\theta}(\cdot \mid s_t)$$

- outputs a distribution,
- exploration is built into the policy,
- enables probability-ratio objectives,
- used by PPO.

Generic actor–critic math

Policy-gradient style actor update

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t$$

Value-critic update

$$\delta_t = r_{t+1} + \gamma(1 - \text{done}_t) V_{\psi}(s_{t+1}) - V_{\psi}(s_t), \quad L_{\text{critic}} = \delta_t^2$$

- The actor is pushed to increase the probability of actions with positive advantage.
- The critic tries to predict future return more accurately.
- DDPG and PPO keep this actor–critic structure, but they differ in policy type, data reuse, and stabilization strategy.

On-policy vs off-policy

On-policy

- learn from data generated by the current policy,
- usually collect fresh trajectories after policy updates,
- examples: REINFORCE, PPO.

Off-policy

- learn from data generated by older policies or another behavior rule,
- can reuse replay-buffer data,
- examples: DQN, DDQN, DDPG.

Why this matters

Off-policy methods can reuse experience more aggressively. On-policy methods often make it easier to reason about policy updates.

Where DDPG and PPO sit in the design space

Aspect	DDPG	PPO
Actor type	Deterministic	Stochastic
Critic type	Action-value $Q(s, a)$	State-value $V(s)$
Data regime	Off-policy	On-policy
Replay buffer	Yes	Usually no long replay buffer
Target networks	Yes	Typically no
Main stability idea	Replay + target nets	Clipped policy update
Action space	Continuous	Often continuous, also discrete

DDPG

Deep Deterministic Policy Gradient

One-sentence idea

DDPG learns a deterministic continuous-action policy and an action-value critic, while using replay memory, target networks, and exploration noise to stabilize learning.

Why DDPG exists

- DQN-style methods need a maximization over actions.
- In continuous control, that maximization is awkward and expensive.
- Deterministic policy gradient methods avoid this by letting the actor output the action directly:

$$a_t = \mu_{\theta}(s_t)$$

Key DDPG idea

Replace “search over actions” with “learn a network that directly proposes the action.” Then train that network with guidance from a critic.

DDPG ingredients at a glance

Learned components

- online actor $\mu_{\theta}(s)$
- online critic $Q_{\phi}(s, a)$
- target actor $\mu_{\theta-}(s)$
- target critic $Q_{\phi-}(s, a)$

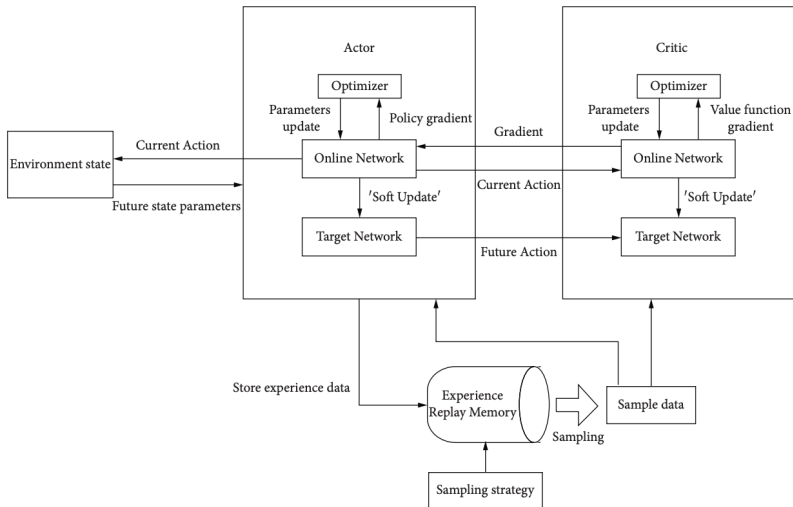
Stability / exploration tools

- replay memory
- random mini-batch sampling
- soft target updates
- exploration noise added to actions

Big picture

Actor chooses the continuous action. Critic evaluates it. Replay and target networks make bootstrapped learning more stable.

DDPG architecture diagram



Notation map for the imported DDPG figure

Figure label	Lecture notation	Meaning
Current Action	a_t	action sent to the environment during interaction
Future state parameters	s_{t+1}	next state from a sampled transition
Future Action	$a_{t+1}^- = \mu_{\theta^-}(s_{t+1})$	action produced by the target actor for the next state
Actor online network	μ_{θ}	actor currently being optimized
Actor target network	μ_{θ^-}	slowly updated copy used in target computation
Critic online network	Q_{ϕ}	critic currently being optimized
Critic target network	Q_{ϕ^-}	slowly updated copy used in target computation

Most important connection in the figure

The **target actor** receives s_{t+1} and outputs a_{t+1}^- . Then the **target critic** receives (s_{t+1}, a_{t+1}^-) and outputs the scalar bootstrap value used in the critic target.

DDPG signal flow I: environment interaction and replay

- 1 The online actor receives the current state s_t and outputs a raw action $\mu_\theta(s_t)$.
- 2 During training, exploration noise is added:

$$a_t = \mu_\theta(s_t) + \varepsilon_t$$

- 3 The environment executes a_t and returns r_{t+1} , s_{t+1} , and done_t .
- 4 The transition $(s_t, a_t, r_{t+1}, s_{t+1}, \text{done}_t)$ is stored in replay memory.
- 5 Random mini-batches are later sampled from replay for learning.

Interpretation

This part of the figure is about **collecting data**. The behavior action is noisy for exploration, but the underlying actor itself remains deterministic.

DDPG signal flow II: target computation and critic update

For a sampled transition $(s_t, a_t, r_{t+1}, s_{t+1}, \text{done}_t)$:

- 1 The **target actor** receives s_{t+1} and outputs the next action

$$a_{t+1}^- = \mu_{\theta^-}(s_{t+1}).$$

- 2 The **target critic** receives (s_{t+1}, a_{t+1}^-) and outputs the bootstrap value $Q_{\phi^-}(s_{t+1}, a_{t+1}^-)$.
- 3 This gives the critic target

$$y_t = r_{t+1} + \gamma(1 - \text{done}_t)Q_{\phi^-}(s_{t+1}, a_{t+1}^-).$$

- 4 The **online critic** receives (s_t, a_t) and is trained to match that target:

$$L_{\text{critic}}(\phi) = \mathbb{E}[(y_t - Q_{\phi}(s_t, a_t))^2].$$

Input-output relation to emphasize in class

Target actor input: $s_{t+1} \Rightarrow$ target actor output: a_{t+1}^-

Target critic input: $(s_{t+1}, a_{t+1}^-) \Rightarrow$ target critic output: one scalar target value.

DDPG signal flow III: actor update and soft updates

- 1 For the actor step, the online actor outputs $\mu_\theta(s_t)$.
- 2 The online critic evaluates that action through $Q_\phi(s_t, \mu_\theta(s_t))$.
- 3 The actor is updated to increase the critic score:

$$J(\theta) = \mathbb{E}[Q_\phi(s, \mu_\theta(s))].$$

- 4 The deterministic policy gradient is

$$\nabla_\theta J(\theta) \approx \mathbb{E}\left[\nabla_a Q_\phi(s, a)|_{a=\mu_\theta(s)} \nabla_\theta \mu_\theta(s)\right].$$

- 5 After the online networks are updated, the target networks move slowly toward them:

$$\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-, \quad \phi^- \leftarrow \tau\phi + (1 - \tau)\phi^-.$$

Important clarification

The target critic does **not** send a training signal back to the target actor. The target branch is only used to build a stable target y_t . The actual actor update uses the **online critic** and **online actor**.

Why does DDPG use online and target networks?

Main idea

DDPG uses **online networks** for active learning and **target networks** for stable target computation.

- **Online actor** $\mu_{\theta}(s)$: the current policy, updated directly by gradient ascent.
- **Online critic** $Q_{\phi}(s, a)$: the current action-value estimator, updated directly by minimizing TD error.
- **Target actor** $\mu_{\theta-}(s)$: a slowly updated copy of the actor, used to generate the next action in the critic target.
- **Target critic** $Q_{\phi-}(s, a)$: a slowly updated copy of the critic, used to generate the next Q-value in the critic target.

Why are target networks needed?

Without target networks, the critic would try to fit a target that changes too quickly, because both the actor and critic are being updated at the same time.

Why do we need target versions of both actor and critic?

The DDPG critic target is

$$y_t = r_{t+1} + \gamma(1 - \text{done}_t) Q_{\phi^-}(s_{t+1}, \mu_{\theta^-}(s_{t+1})).$$

What is inside this target?

The target depends on two learned quantities:

- the **next action** from the target actor: $\mu_{\theta^-}(s_{t+1})$,
 - the **next Q-value** from the target critic: $Q_{\phi^-}(s_{t+1}, \mu_{\theta^-}(s_{t+1}))$.
-
- If we used the **online actor** here, the target would move every time the actor changes.
 - If we used the **online critic** here, the target would move every time the critic changes.
 - Therefore, DDPG uses **target actor + target critic** so that both parts of the target change slowly.

Takeaway

Online networks learn fast. Target networks move slowly. This makes the Bellman target more stable and helps DDPG training converge.

DDPG critic target and loss

For a sampled transition $(s_t, a_t, r_{t+1}, s_{t+1}, \text{done}_t)$, the target actor proposes the next action:

$$a_{t+1}^- = \mu_{\theta^-}(s_{t+1})$$

Then the target critic gives the bootstrap target:

$$y_t = r_{t+1} + \gamma(1 - \text{done}_t)Q_{\phi^-}(s_{t+1}, a_{t+1}^-)$$

The critic minimizes:

$$L_{\text{critic}}(\phi) = \mathbb{E}\left[(y_t - Q_{\phi}(s_t, a_t))^2\right]$$

Interpretation

The critic is learning Bellman consistency: the value of the current state-action pair should match immediate reward plus discounted estimated future value.

DDPG actor update: what is the actor trying to do?

The actor wants to choose actions that the critic considers valuable:

$$J(\theta) = \mathbb{E}[Q_\phi(s, \mu_\theta(s))]$$

The deterministic policy gradient has the form

$$\nabla_\theta J(\theta) \approx \mathbb{E}\left[\nabla_a Q_\phi(s, a)|_{a=\mu_\theta(s)} \nabla_\theta \mu_\theta(s)\right]$$

Intuition

The critic tells us how the value changes when the action changes slightly. The actor then adjusts its parameters so that it outputs actions with larger critic value.

DDPG actor update: plain-language explanation

- The actor does **not** receive a supervised label saying “the correct action is 0.42.”
- Instead, it proposes an action and asks the critic: *how good is this action in this state?*
- If the critic score increases when the action moves in a certain direction, the actor is nudged in that direction.
- Over time, the actor learns to output actions that the critic scores highly.

Mental model

Critic = learned evaluator of continuous actions.

Actor = network that learns to move toward actions the critic prefers.

Why DDPG needs replay memory

- Consecutive transitions are highly correlated.
- Neural networks are easier to train with more i.i.d.-like mini-batches.
- Replay memory stores many past transitions and later samples them randomly.

Benefits of replay

- breaks temporal correlation,
- improves sample reuse,
- makes off-policy learning practical,
- gives the critic and actor broader state-action coverage.

Why DDPG needs exploration noise

A deterministic actor always outputs the same action for the same state. Therefore, it does not explore by itself.

Data-collection rule

$$a_t = \mu_{\theta}(s_t) + \varepsilon_t$$

- ε_t is often Gaussian noise or Ornstein-Uhlenbeck noise.
- The noise is used only when collecting experience from the environment.
- The learned actor itself remains deterministic.

Key idea

In DDPG, exploration is not inside the policy distribution. It is injected externally by adding noise to the actor output.

DDPG training loop

- 1 Observe the current state s_t .
- 2 Compute a noisy behavior action $a_t = \mu_\theta(s_t) + \varepsilon_t$.
- 3 Execute the action and observe r_{t+1} , s_{t+1} , and done_t .
- 4 Store the transition in replay memory.
- 5 Sample a random mini-batch from replay.
- 6 Update the critic using the target y_t .
- 7 Update the actor using the online critic gradient.
- 8 Soft-update the target actor and target critic.

Strengths and limitations of DDPG

Why DDPG is useful

- handles continuous actions directly,
- reuses data via replay memory,
- extends actor-critic naturally from Module 3,
- often more sample-efficient than purely on-policy methods.

What can go wrong

- training can be brittle,
- exploration quality matters a lot,
- critic errors can mislead the actor,
- performance can be sensitive to hyperparameters.

Practical lesson

DDPG is conceptually elegant, but stable training requires several engineering tricks to work together.

PPO

Proximal Policy Optimization

One-sentence idea

PPO is a stochastic actor–critic method that improves the policy cautiously by clipping the actor update, so the new policy does not move too far from the old one in one optimization round.

Why PPO exists

Plain policy-gradient updates can be unstable.

- A batch of trajectories may suggest a very large parameter update.
- If the policy changes too much in one step, performance can collapse.
- Earlier trust-region methods tried to constrain this change, but they are more complex to implement.

PPO design goal

Improve the policy, but keep the new policy **close enough** to the old policy that learning remains stable.

PPO ingredients at a glance

Learned components

- stochastic actor $\pi_{\theta}(a | s)$
- value critic $V_{\psi}(s)$
- old policy snapshot $\pi_{\theta_{\text{old}}}$

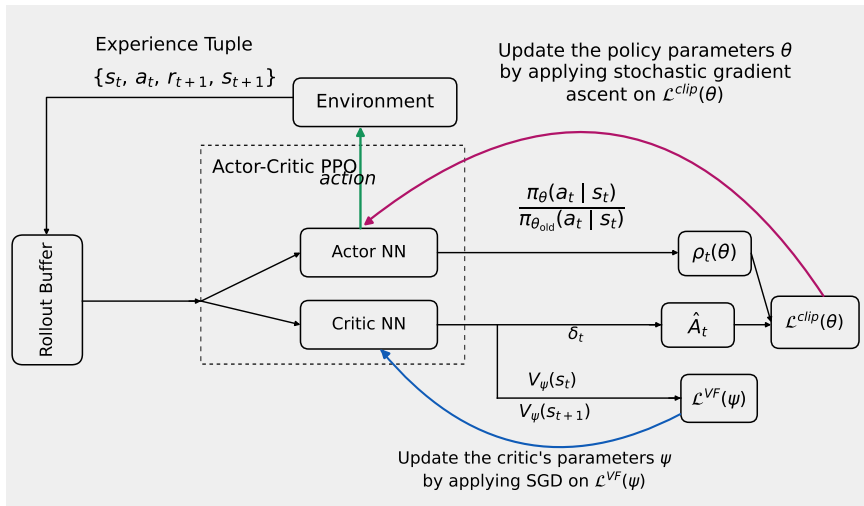
Key learning signals

- importance ratio $\rho_t(\theta)$
- advantage estimate $\hat{A}_t$
- clipped actor objective
- value-function loss
- optional entropy bonus

Big picture

Collect fresh rollouts with the current policy, estimate which sampled actions were better or worse than expected, and then improve the policy without letting it move too far in one update round.

PPO architecture diagram



PPO signal flow I: rollout collection and old-policy snapshot

- 1 The current actor samples actions from $\pi_{\theta}(\cdot \mid s_t)$.
- 2 The environment returns a transition $(s_t, a_t, r_{t+1}, s_{t+1}, \text{done}_t)$.
- 3 These transitions are stored in a temporary **rollout buffer** for the current update round.
- 4 Before optimization, we freeze a copy of the current policy as $\pi_{\theta_{\text{old}}}$.
- 5 During a few optimization epochs, the collected batch stays fixed while $\pi_{\theta_{\text{old}}}$ stays fixed as the reference distribution.

Important clarification

This buffer is usually a **fresh on-policy rollout buffer**, not a long replay memory like in DDPG.

PPO signal flow II: critic, returns, and advantage

Critic output

$$V_{\psi}(s_t)$$

The value critic predicts the expected return from the current state.

One-step TD error

$$\delta_t = r_{t+1} + \gamma(1 - \text{done}_t)V_{\psi}(s_{t+1}) - V_{\psi}(s_t)$$

Advantage signal

$$\hat{A}_t \approx Q(s_t, a_t) - V(s_t)$$

In practice, $\hat{A}_t$ is often estimated from temporal difference (TD) information or with generalized advantage estimation (GAE).

Interpretation

- $\hat{A}_t > 0$: the action was better than the critic expected.
- $\hat{A}_t < 0$: the action was worse than the critic expected.
- $\hat{A}_t \approx 0$: the action was about as good as expected.

Estimating advantage with TD and GAE

TD-style advantage estimate

For a one-step bootstrap estimate,

$$\hat{A}_t^{\text{TD}(0)} = \delta_t = r_{t+1} + \gamma(1 - \text{done}_t)V_\psi(s_{t+1}) - V_\psi(s_t)$$

- Uses the critic at the current and next state.
- Simple and low-variance, but only one-step look-ahead.

GAE: Generalized Advantage Estimation

GAE combines many future TD errors:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$$

with

$$\delta_t = r_{t+1} + \gamma(1 - \text{done}_t)V_\psi(s_{t+1}) - V_\psi(s_t)$$

- $\lambda = 0$: close to one-step TD.
- $\lambda \rightarrow 1$: uses longer-horizon information.

Connection to PPO figure

The **V-function block** provides $V_\psi(s_t)$, the rollout buffer provides rewards and next states, and the **advantage block** turns TD errors into $\hat{A}_t$ using TD(0) or GAE.

Why PPO compares new and old policy probabilities

PPO measures how much the current policy has changed on the sampled action:

$$\rho_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

If $\rho_t(\theta) = 1$

The new policy gives the sampled action the same probability as before.

If $\rho_t(\theta) > 1$

The new policy makes that sampled action more likely.

If $\rho_t(\theta) < 1$

The new policy makes that sampled action less likely.

Role in the figure

The ratio block connects the **current actor** and the **old policy snapshot**. It tells the optimizer whether the new actor is drifting too far from the policy that generated the batch.

PPO actor loss: term-by-term explanation

The clipped PPO actor objective is

$$L^{\text{clip}}(\theta) = \mathbb{E} \left[\min (\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right]$$

Meaning of each part

- $\pi_{\theta}(a_t | s_t)$: probability of the sampled action under the **current** actor.
- $\pi_{\theta_{\text{old}}}(a_t | s_t)$: probability of the same action under the **old** policy that generated the batch.
- $\rho_t(\theta)$: how much the action probability changed.
- $\hat{A}_t$: whether that sampled action was better or worse than expected.
- $\text{clip}(\cdot)$: limits how far the ratio can move away from 1.
- $\min(\cdot, \cdot)$: takes the more conservative improvement signal.

PPO clipping intuition: positive vs negative advantage

Case 1: $\hat{A}_t > 0$

The action was better than expected.

PPO wants to make it more likely, but **not too much more likely**.

If the ratio becomes too large, clipping removes extra incentive to increase it further.

Case 2: $\hat{A}_t < 0$

The action was worse than expected.

PPO wants to make it less likely, but **not too aggressively**.

If the ratio becomes too small, clipping limits the penalty.

Main idea

Move the policy in the right direction, but keep each update within a safer neighborhood of the old policy.

How the loss changes the actor

If $\hat{A}_t > 0$

The sampled action was good. PPO tries to increase its probability under the new policy, but only up to the clipping threshold.

If $\hat{A}_t < 0$

The sampled action was poor. PPO tries to decrease its probability, but only up to the clipping threshold.

Why this is safer than a plain policy gradient

A plain policy gradient would keep pushing as long as the gradient points in the same direction. PPO stops rewarding an overly large probability change on the same batch.

PPO value loss, entropy bonus, and total objective

Besides the actor objective, PPO also trains a value critic and may reward policy uncertainty.

Typical value loss

$$L^{VF}(\psi) = \mathbb{E}[(V_{\psi}(s_t) - \hat{R}_t)^2]$$

Typical combined training loss (minimization form)

$$L_{\text{total}}(\theta, \psi) = -L^{\text{clip}}(\theta) + c_v L^{VF}(\psi) - c_e \mathcal{H}(\pi_{\theta}(\cdot | s_t))$$

- L^{clip} : improves the actor conservatively.
- L^{VF} : makes the critic predict return targets more accurately.
- $\mathcal{H}(\pi_{\theta})$: entropy of the action distribution; higher entropy means a more spread-out, less certain policy.

How to read the combined PPO loss

Term-by-term meaning

- $-L^{\text{clip}}(\theta)$: actor term.
In minimization form, this means we are really trying to **maximize** the clipped PPO objective.
- $c_v L^{VF}(\psi)$: critic term.
This trains the value network so that $V_\psi(s_t)$ matches the return target $\hat{R}_t$.
- $-c_e \mathcal{H}(\pi_\theta)$: entropy term.
In minimization form, this encourages **larger entropy**, so the policy does not become too certain too early.

Why one total loss?

The optimizer needs one scalar objective. PPO therefore combines:

- **policy improvement**,
- **value-function fitting**,
- **exploration encouragement**.

Role of the weights

- c_v : how strongly we care about critic fitting relative to actor improvement.
- c_e : how strongly we encourage exploration through policy entropy.

Interpretation of the tradeoff

- If c_v is too small, the critic may be weak and the advantage estimates may become noisy.
- If c_v is too large, optimization may focus too much on value fitting and not enough on policy improvement.
- If c_e is too small, the policy can collapse too early into overconfident behavior.
- If c_e is too large, the policy may stay too random for too long.

Signal flow when PPO uses the combined loss

Forward signal flow

- Rollout buffer provides $(s_t, a_t, r_{t+1}, s_{t+1}, \text{done}_t)$.
- Current policy computes $\pi_\theta(a_t | s_t)$; frozen old policy provides $\pi_{\theta_{\text{old}}}(a_t | s_t)$.
- Their comparison gives $\rho_t(\theta)$.
- The value network outputs $V_\psi(s_t)$ and supports the construction of $\hat{A}_t$ and $\hat{R}_t$.
- These signals meet in the total loss:

$$L_{\text{total}} = -L^{\text{clip}}(\theta) + c_v L^{\text{VF}}(\psi) - c_e \mathcal{H}(\pi_\theta).$$

Backward signal flow

$$\nabla_\theta L_{\text{total}} = -\nabla_\theta L^{\text{clip}} - c_e \nabla_\theta \mathcal{H}(\pi_\theta), \quad \nabla_\psi L_{\text{total}} = c_v \nabla_\psi L^{\text{VF}}.$$

Which parameters are updated?

- Actor parameters θ : updated by the clipped actor term and the entropy term.
- Critic parameters ψ : updated by the value loss.
- Old policy snapshot: **not** optimized; it is only a fixed reference for the ratio.

Key teaching message

The critic does not directly choose actions. It improves the actor's learning signal by producing better value estimates, which lead to better advantage estimates.

What exactly is the return target $\hat{R}_t$?

Meaning

$$V_{\psi}(s_t) \approx \hat{R}_t$$

$\hat{R}_t$ is the **training target for the value critic**. It is the estimated return that the critic should predict for state s_t .

Typical construction in PPO

A common implementation builds it from the value estimate and the advantage:

$$\hat{R}_t = \hat{A}_t + V_{\psi}(s_t)$$

If $\hat{A}_t$ comes from GAE, then $\hat{R}_t$ becomes a bootstrapped multi-step return target.

Interpretation

- r_{t+1} is one immediate reward.
- $\hat{R}_t$ summarizes the **future reward signal** from time t onward.
- The critic is trained so its state value matches that target.

How the value loss fits into the PPO architecture

Flow through the architecture

- 1 The rollout buffer provides states, rewards, next states, and terminal flags.
- 2 The V-function block outputs $V_{\psi}(s_t)$ and $V_{\psi}(s_{t+1})$.
- 3 TD or GAE produces $\hat{A}_t$.
- 4 The return target is formed as $\hat{R}_t = \hat{A}_t + V_{\psi}(s_t)$.
- 5 The loss block compares $V_{\psi}(s_t)$ against $\hat{R}_t$ using L^{VF} .

Why this matters

Without the value loss, the critic would not learn a useful baseline. Then the advantage estimates become noisier, and the actor update becomes less reliable.

Key role of the critic

The critic is not choosing actions. It is supplying a better learning signal for the actor.

What does entropy mean in PPO, and why do we need it?

Entropy of the policy

For a stochastic actor, entropy measures how **spread out** the action distribution is.

- **High entropy**: the policy keeps several actions plausible.
- **Low entropy**: the policy becomes very certain and concentrated.

Why include $-c_e \mathcal{H}(\pi_\theta)$ in the minimization loss?

Because minimizing L_{total} with a negative entropy term encourages **larger entropy**. This delays premature collapse into an overly certain policy.

Why it helps

- promotes exploration,
- reduces the risk of getting stuck too early,
- is especially useful early in training.

Tradeoff

Too little entropy bonus can make the policy collapse too early. Too much can keep the policy overly random.

How the combined PPO loss maps to the architecture

Three contributions inside the loss block

- 1 **Actor term:** $-L^{\text{clip}}(\theta)$ uses the ratio block and advantage block.
- 2 **Critic term:** $c_v L^{\text{VF}}(\psi)$ uses the V-function output and the return target $\hat{R}_t$.
- 3 **Entropy term:** $-c_e \mathcal{H}(\pi_\theta)$ uses the current policy distribution itself.

Parameter view

- Actor parameters θ are influenced by the clipped objective and entropy term.
- Critic parameters ψ are influenced by the value loss.
- The old policy is only a **reference**; it is not optimized.

Clean reading of the figure

The loss block is where the architecture merges:

- policy comparison,
- advantage estimation,
- value fitting,
- exploration encouragement.

Reading the PPO figure block by block

- ① **Policy network**: samples actions and later recomputes action probabilities for the collected batch.
- ② **Old policy**: frozen snapshot used only as a reference in the ratio.
- ③ **V-function**: estimates state values for return / advantage computation.
- ④ **Advantage block**: tells the actor whether sampled actions were better or worse than expected.
- ⑤ **Loss function**: combines the clipped actor objective with the value loss, and often an entropy term.

Student takeaway

PPO has two learning tasks at once: a **conservative actor update** and a **value-function fit**. The old policy appears only to control how large the actor update is.

PPO training loop

- ① Run the current policy in the environment and collect a batch of trajectories.
- ② Estimate returns and advantages from that fresh data.
- ③ Freeze the current policy as the “old” policy reference.
- ④ Update the actor for a few epochs using the clipped objective.
- ⑤ Update the critic using the value loss on the same batch.
- ⑥ Collect fresh data again with the updated policy.

Rhythm of PPO

Collect fresh data → improve policy cautiously → collect fresh data again.

Why PPO is usually easier to stabilize than naive policy gradients

- The actor update is constrained indirectly through clipping.
- The critic provides a lower-variance learning signal than raw returns alone.
- Multiple mini-batch epochs can be performed on the same fresh rollout batch.
- An entropy bonus can keep exploration alive while the policy is still uncertain.

But note the tradeoff

PPO is usually less sample-efficient than off-policy replay-based methods because it keeps collecting fresh data instead of learning from a large long-term replay memory.

Strengths and limitations of PPO

Why PPO is useful

- conceptually clean,
- typically stable in practice,
- works with stochastic policies,
- widely used as a general-purpose baseline.

What to keep in mind

- usually needs fresh on-policy data,
- can be less sample-efficient than replay-based methods,
- still depends on good advantage and value estimates,
- clipping helps stability but does not make learning automatic.

DDPG vs PPO: side-by-side comparison

Aspect	DDPG	PPO
Policy type	Deterministic actor $\mu(s)$	Stochastic actor $\pi(a s)$
Critic type	Action-value $Q(s, a)$	State-value $V(s)$
Data regime	Off-policy, replay-based	On-policy, fresh rollout based
Exploration	Added action noise	Built into policy stochasticity
Main stabilization trick	Replay + target networks	Clipped policy ratio
Typical teaching message	Direct continuous-action control	Safe policy improvement

Big conceptual contrast

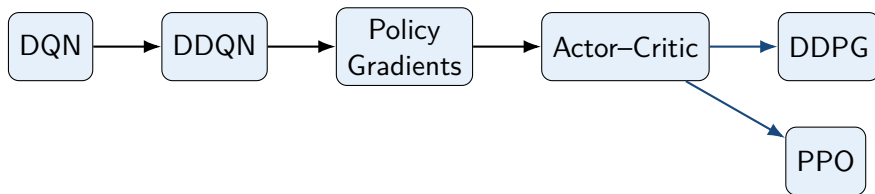
DDPG says: *output the continuous action directly and learn from replay.*

PPO says: *keep a stochastic policy, but update it conservatively.*

What should students remember after this module?

- Actor–critic means the actor chooses actions and the critic evaluates them.
- Continuous control motivates direct policy output instead of exhaustive action maximization.
- DDPG is deterministic, off-policy, replay-based, and uses target networks.
- PPO is stochastic, on-policy, and stabilizes learning by clipping policy updates.
- In PPO, the value loss trains the critic, and the entropy bonus can help maintain useful exploration.
- Both methods are actor–critic, but they solve the stability and exploration problems in very different ways.

Conceptual map: from Module 3 to Module 4



Course continuity

Module 3 built the bridge from value-based deep RL to actor-critic thinking. Module 4 uses that bridge to introduce two modern continuous-control families built on top of it.

Suggested reading

- Sutton and Barto, *Reinforcement Learning: An Introduction*, 2nd ed., 2018.
- Silver et al., *Deterministic Policy Gradient Algorithms*, ICML 2014.
- Lillicrap et al., *Continuous Control with Deep Reinforcement Learning*, ICLR 2016 / arXiv 2015.
- Schulman et al., *Proximal Policy Optimization Algorithms*, arXiv 2017.
- OpenAI Spinning Up documentation for DDPG and PPO.